Assignment 8: The Botnet Saves the World

Teachers: Fatima Zahra Errounda, Stephan Schiffel

Document last updated: October 19, 2026

Task and Logistics

Your task is to extend the server you wrote in Assignment 6 into a store-and-forward message relay: it exchanges messages with other groups' servers, passes on messages for groups further away, and holds messages for groups that are currently unreachable. **The focus is on the communication *between servers*.** Everything from Assignment 6 is assumed in place and reused; this document describes only what is new.

- A group assignment, for groups of two or three students.
- **Register your group on the TSAM server in the first few days** to pick the name your server goes by — see **Your Group Identity**.
- C or C++, graded on Linux (much like the TSAM server). Develop on whatever you like, but check it compiles and runs on Linux before submitting.
- **No third-party libraries for networking or socket handling** — use the socket API directly. Libraries for other purposes are fine if you include them in your submission. If in doubt, ask!
- **Start early.** Get your server talking to another group's in the first few days and leave it running: several bonus parts need evidence from different days, which you cannot produce retroactively.
- **Write your own code.** Use an AI assistant to *understand* things, not to produce your submission — code you cannot talk through is worth nothing to you in the exam.
- **Co-operate.** Co-ordinating servers, agreeing what the protocol means and testing against each other are all expected.

Scenario

Newsflash 2030.

As a result of the 2022 Hunga Tonga eruption increasing stratospheric water content by 10%, and the resulting rapid increase in global average temperatures during the rest of the decade, Iceland is enjoying a much improved climate and increasing economic strength. However, as the fighting on the eastern front moves into its 8th year, and Russian troops encircle Budapest, the struggle on the Internet intensifies. A few days ago major cyber hostilities broke out between the four superpowers. Three hours ago all social media went off the Internet, the mobile telephone network failed, and email and all other forms of communication have become extremely unreliable. What traffic still moves is being read: the superpowers are intercepting the League's communications wholesale, which is why a new network is needed rather than a repair of the old one. In Iceland the medical system has resorted to communication through bicycle couriers, and all financial transactions are currently suspended.

As a founding member of the League of Little Nations, Iceland has been asked by the Duchy of Grand Fenwick to lead the rapid development of alternative and robust peer-to-peer forms of communication for the citizens of the Northern Democratic Alliance. With the priority being to get any form of simple messaging up and running, they have provided a poorly written basic specification for the protocol it should use.

In order to build this network, you will need to give some thought to routing commands through the network, connecting to and leaving the botnet, storing messages for disconnected servers, message expiry, and handling commands for other groups. Remember also that there is other information besides that purely in the messages — for example, knowing which link to your server the message arrived on, and which server is at the other end of that link.

Beware: the superpowers have gotten wind of the League's plan, and they are already inside the network. Expect hostile servers to join the botnet under forged identities, to impersonate groups you have come to trust, to tamper with or replay the messages they are asked to relay, to harvest everything that passes through them, and to flood your server until it is driven off the network. Nothing in the protocol below will tell you who you are really talking to. Assume nothing about a peer that its own behaviour has not shown you — and keep the evidence, because you will be asked for it.

Rules of Engagement

0. Don't crash the (bot) network.
1. Don't crash the campus network either.
2. "Be strict in what you send, be tolerant in what you receive!" Your server must not crash, hang or spin on input that does not follow the protocol, no matter which server it came from. Being tolerant does not mean putting up with anything: You are allowed to (temporarily) block addresses if a server there is misbehaving.
3. Your server must identify itself within the botnet using the group ID you chose when you registered. Student groups are named `G_…`, instructor servers are `I_…`.

   If you need somewhere to start, the first of our servers listens on `130.208.246.98:5001`. That is the only address you may hard-code.
4. TCP ports 4000–4200 are available on the TSAM server (`130.208.246.98`) for external connections.
5. Your client must connect to your server only, and issue any commands to other servers on the botnet via your server.
6. You are encouraged to reach out to other groups to be on the botnet with them at the same time, and co-operate in getting things working.
7. **Noticing bad behaviour is the assignment; producing it is not.** Do not impersonate another group, flood another group's server, or try to gain access to any machine or account. Blocking a misbehaving address (Rule 2) is the sanctioned response.

See **Hints** at the end for how to keep a server running on the TSAM server after you log out, how to let your group members read each other's files, and how to ask one of our servers to connect to you.

Your Group Identity

Every server on the botnet has a group ID, and you need to choose yours.

Log in to the TSAM server and run `tsam-register` with the name you want, followed by the usernames of *all* the members of your group, including your own:

```
$ tsam-register --check banana          # is it free? (optional)
G_banana is available

$ tsam-register banana alice bob carol
Registered. This is your group's botnet identity:

  group ID : G_banana
  members  : alice, bob, carol

wrote identity.h
```

The name must match `[A-Za-z0-9-]{1,12}`. That is narrower than you might expect, and for the same reason every real protocol restricts its identifiers: your group ID travels through the space-separated fields of the log, the commas in `HELO` and `SENDMSG`, the semicolons in `SERVERS`, the colon in `STATUSRESP`, and a C string literal in `identity.h`. A name containing any of those characters could break the parsers of *every other group that talked to you*, and you would look like the server at fault.

- Your TSAM-server login is how the service knows who you are, so there is no password and you cannot register as somebody else.
- Only one member of a group needs to register. Any member can run `tsam-register` and gets *the same* group ID back.

`identity.h`

Registering also writes `identity.h`, which is where your server takes its group ID from. Keep the file exactly as written, next to your other source files, and `#include "identity.h"` to use the macro it defines. **Do not hard-code your group ID in any other file.** The reason is that **we may replace `identity.h` with a test identity and rebuild your code** when grading and testing your server. If the group ID is duplicated or comes from somewhere else, that will not work, and you will lose the points that depend on it.

Server Specification

All requirements from Assignment 6 continue to apply, in particular: the server operates autonomously and never reads from the terminal, it handles many connections simultaneously and sequences of commands per connection, it keeps a timestamped log of all commands sent and received (in the format fixed in **Server Log** below), it does not busy-wait (CPU usage essentially zero while idle), it handles malformed input without crashing, and it hard-codes no IP addresses or port numbers.

In addition:

- Your server must listen on one TCP port for *other servers* as well as your own client to connect to it. A single listening port serves both; it is up to your server to work out what is at the other end of a connection.
- You may run the server on the TSAM server or on some other computer, but it must be actually reachable for other groups to connect to it.
- The executable of your server must be called `tsamserver`, and the TCP port it listens on is its **only required command line argument**:

  ```
  ./tsamserver 4044
  ```

  Any further command line arguments you want to define must be optional, and your server must function in a test without them.
- While part of the botnet, your server must try to maintain connections to at least 3 and at most 8 other servers. It must try to re-establish connections that have been lost and/or find new peers. Connections to other groups' servers must be prioritised over instructor servers.

Server Log

Your server must write its log to a file called `server.log`. Unlike in Assignment 6, **the format of the log lines is fixed**: we read these logs automatically, and cross-check your log against the logs of the groups you talked to.

You are given `log_event.h` with this assignment, and it writes the format for you. Put it next to your own source files, include it, and call one of its three functions:

```
log_event("START", NULL, 0, GROUP_ID, "listening on port 4001");
log_event("ACCEPT", ip, port, NULL, "incoming connection");
log_event_bytes("RECV", ip, port, peer_id, cmd, cmd_len);
log_note("no route to %s right now, holding %d messages for it", id, n);
```

It appends to `server.log`, writes the timestamp, substitutes `-` for any `NULL`, and escapes the detail so that message content cannot break a line. Use `log_event_bytes` for the commands you `RECV` and `SEND` — message content may contain NUL bytes, which the string form cuts short — and `log_event` for everything else. **The format, a worked example and the escaping rules are in the comments at the top of `log_event.h` — read those.**

What matters in this document:

- A line recording a protocol event has **five space-separated fields**:

  ```
  <TIMESTAMP> <EVENT> <IP:PORT> <GROUP_ID> <detail>
  ```

  The timestamp is **ISO 8601, in UTC**, and `<EVENT>` is one of `START`, `ACCEPT`, `CONN`, `RECV`, `SEND`, `DROP` or `CLOSE`. A field whose value you do not know is written as a single `-`.

- **Log more than the bare protocol events** — what your server decided and why, what it is holding, when it gave up on something. Worth points under 1(f), and your best debugging tool. Such lines may say anything, but must start with a timestamp; `log_note` writes them as `<TIMESTAMP> note <your text>`.

- You may **replace or modify** `log_event.h` and write the log yourself, but **the format is not negotiable** — what your server writes must match it exactly. We read these logs automatically, and **a file we cannot parse costs you the points it was evidence for**.

Keeping the log

Your log is the evidence for most of the points below, and **it cannot be reconstructed afterwards**. `log_event.h` appends, so a restart adds to the file — that is what the `START` line is for. The rest is up to you:

- **Keep one log for the whole assignment**, covering every run. If you change machines, concatenate the two in chronological order.
- **Back it up regularly.** It is worth more than your code.
- **Watch its size** — you share a disk with the whole class.
- **If you lose part of it, say so in your `README.md`** — we would rather read that than guess. But **points whose evidence is missing cannot be awarded**.

Interim log uploads. There is a separate assignment on Canvas, next to this one, where you can upload your `server.log` while you are still working — as many times as you like, gzipped if it is large. Two things happen to it:

- **It is stored**, so a copy is safe with us if you lose yours.
- **It is checked**, and you can read the result straight away: whether the format conforms, which lines did not parse, and whether anything is obviously missing.

A format problem takes half an hour to fix in week one; at the deadline it cannot be fixed at all.

A peer may flood your log. A server stuck in a reconnect loop, or one forwarding every message to all its peers, can bury you in lines. This happens every year, almost always by accident.

- You may **summarise repeated events**: log the first, then a `log_note` line from time to time saying how many you left out and over what period.
- **Key that on the IP address, not the group ID.** A peer in a reconnect loop never reaches `HELO`, so you have no group ID for it, and its source port changes every time.
- You may also stop talking to it, as Rule 2 allows.
- Summarising *as you go* is fine; editing your log down *afterwards* is what **Submission** forbids. Say in your `README.md` what you suppress.

Server Communication

Commands between servers use **exactly the same framing as in Assignment 6**:

`<SOH><length><STX><command><ETX>`

As before, your server must support commands up to a length of 5000 bytes and may discard any commands that are longer. Remember that TCP is a byte stream: a frame may arrive split over several `recv()` calls, and several frames may arrive in one. This matters more here than it did in Assignment 6, because you no longer control the server at the other end.

The protocol below is a draft. It was written in a hurry by the League of Little Nations and it is **incomplete and in places ambiguous**. This is on purpose, and it is part of the assignment: a real protocol only works if the people implementing it agree on what it means. It also assumes everybody on the network is co-operating. They are not.

Talk to the other groups early, test against each other, and expect to revise your implementation as the botnet grows — that is the normal life of a protocol, not a sign that something has gone wrong. Document the choices and assumptions you ended up with in your `README.md`. If you are unsure whether something is in the spirit of the protocol, ask.

Server Commands

Your server must support at least the following commands in communicating with other servers. Other commands are allowed if they facilitate network communication.

Command	Expected behaviour
`HELO,<FROM_GROUP_ID>`	Reply with the `SERVERS` response (below). **This must be the first command each end sends on a connection**: send it directly after `connect()` and `accept()`.
`SERVERS,<server list>`	Response to `HELO`: a semicolon-separated list of the *directly connected* servers of the responding server. Each server is specified as `<GROUP_ID>,<IP>,<PORT>` with the host IP and the port they accept connections on.
Example: `SERVERS,G_banana,130.208.246.98,4001;G_pelican,130.208.246.98,4002;`	
`KEEPALIVE,<No. of messages>`	Periodic message to 1-hop connected servers, indicating the no. of messages the server sending the `KEEPALIVE` message has waiting for the server at the other end. Do not send more than once a minute.
`GETMSGS,<GROUP_ID>`	Get messages for the specified group in the form of `SENDMSG` commands.
`SENDMSG,<FROM_GROUP_ID>,<TO_GROUP_ID>,<TIMESTAMP>,<message content>`	Send a message to another group. `<TIMESTAMP>` is the time when the message was first created as seconds since the Unix epoch (UTC).
Example: `SENDMSG,G_banana,G_pelican,1792915407,Hello, world`	
`STATUSREQ`	Reply with `STATUSRESP` as below.
`STATUSRESP,<name1>:<count1>,<name2>:<count2>,...`	Response to `STATUSREQ`: the servers you are holding messages for and how many, listing only servers with at least one stored message. If you are holding no messages at all, the response is `STATUSRESP,EMPTY`.
Example: `STATUSRESP,G_tapir:20,I_oracle:2`	

Client-Server Protocol

The client specification from Assignment 6 applies unchanged:

- group IDs take the place of user names
- `STATUSREQ` now reports the messages your server holds for other servers/groups instead of users

Note that the client's `SENDMSG` has no timestamp field: your own server needs to fill that in when it gets the message from the client.

You may implement additional client commands if you wish; they are a good way to inspect or configure your server at run-time — for example, to tell it which server to connect to, or to list the servers it is currently connected to.

Submission

Submit a single zip file containing all your source code and the following files:

```
Makefile
README.md
identity.h
server.log
```

- `Makefile`: running `make` in the submission directory, with no arguments and no manual setup, must build both `tsamserver` and `tsamclient`.
- `README.md` must describe:
  - how to compile and run your code (optional parameters and/or configuration),
  - which operating system you developed and tested on,
  - the commands implemented by your client and server,
  - the behaviour of your server in response to the commands and to other relevant events. You should especially describe any additional commands or behaviour that are not already described in this assignment.
  - **which of the points and bonus points below you expect to get**, and where the evidence for each one is. **Provide the timestamps** of the relevant lines in `server.log`.
  - **a statement of authorship**: who wrote which parts, and any tool or source you used and what for. Using an AI assistant to understand something belongs here and is fine; having one produce your server, `README.md` or `server.log` is not permitted, and claiming otherwise here is academic dishonesty.
- `identity.h`: your group identity, exactly as created by `tsam-register`.
- `server.log`: **your complete server log** for the whole assignment. We need the whole file because we cross-check logs between groups. **Do not remove, reorder or tidy up lines.** If your complete log is very large (more than 100 MB), you may compress it (`server.log.gz` is acceptable). Alternatively, ask for instructions on how to handle this.
- The zip must **not** contain hg/git/etc. repositories, `__MACOSX` or related files/folders, compiled binaries/object files, or other hidden files.

Grading

For the points below that are evidenced by your log, point us at the relevant lines in `server.log` from your `README.md` **by timestamp**. A claim with no timestamps is one we may not be able to find the evidence for, and **points whose evidence we cannot find cannot be awarded**.

1. **(a)** (50 points) Implement the server-to-server protocol described above, on top of your Assignment 6 client and server. This covers both halves of it:
   - **every command** — sent at the right time, and handled correctly on receipt.
   - **the behaviour around them** — joining the botnet under your registered group ID, staying connected, contributing to message delivery, and continuing to work when peers disconnect or send nonsense.

   Your server must issue some commands on its own (e.g. time-based) as well as responding to the client and other servers.

   If the class agrees to change or extend part of the protocol, say so in your `README.md` and we will mark what you agreed rather than what is written above.

   (b) (10 points) Have been successfully connected to by someone else's server. That means the other server needs to initiate the connection (call `connect`) and your server accept it (call `accept`). Give the timestamps of the `ACCEPT` line and of **all four** of the `HELO` and `SERVERS` lines that followed it — theirs and yours, in both directions.

   (c) (10 points) Successfully receive **messages** from at least 2 other groups. A message means a `SENDMSG` addressed to your group — not any other command, and not a `SENDMSG` merely passing through on its way to somebody else. A message only counts as received once **your client has fetched it from your server**, so your log has to show both halves: the `SENDMSG` arriving, and the message going on to your client. Give the timestamps of both lines, for each of the two groups.

   (d) (10 points) Successfully send **messages** to at least 2 other groups. Again a `SENDMSG` your group originated, rather than one you relayed for somebody else. Give the timestamps of the lines where your client handed each message to your server and where your server sent it on.

   (e) (10 points) Your submission follows the format described in the **Submission** section above, your log follows the format in **Server Log**. In particular, **nothing**

may be hard-coded: no group IDs (other than your own in `identity.h`), and no IP addresses or port numbers — with the single exception of our first server, `130.208.246.98:5001`, which you may hard-code as a starting point for joining the botnet.

(f) (10 points) Code is well structured and documented. The server's log, beyond the bare protocol events, contains useful information about the internal state and behaviour of the server.

(g) If you fixed any bugs since Assignment 6, you can have your Assignment 6 grade improved, since the code for this assignment should mostly work for Assignment 6 as well. There is a separate assignment on Canvas, next to this one, with the details — the submission above does not claim it for you, so go there if you want it.

Bonus Points

The maximum grade for the assignment is 100 points. Up to **100 bonus points** can be awarded additionally — essentially a whole missed assignment's worth — and the parts below add up to more than that on purpose, so pick the ones you want to go for. The bonus points carry over to other assignments and the final grade (but do not count towards the passing threshold for the final exam).

List in your `README.md` the bonus points you want to claim, with the timestamps that show you are eligible. Several of these need evidence collected at different times over the two weeks, which you cannot produce retroactively — read them early and decide what you are going for.

2. **(a)** (10 points) **Early Bot:** Submit a working server-to-server implementation within the first week: your server connects to at least one other server, exchanges `HELO`/`SERVERS` with it, and delivers a message to it. The remaining commands do not need to be finished yet, and you can improve the code later for the final submission. Submit the code together with the log of the exchange.

   (b) (10 points) **Ready Bot:** Your server connected to the botnet *and responding* on at least 5 days for at least 2 hours each. Point at commands received from other servers on each of those days; the first timestamp of each 2-hour window is enough.

   (c) (10 points) **Chatty Bot:** Reaching a good part of the rest of the class: **5 points for messages** (**not** commands!) **received from 10 different groups, 10 points for 20 different groups**. List the groups in your `README.md` with a timestamp for each. Instructor servers (`I_…`) do not count — this part is about reaching the rest of the class.

   (d) (20 points) **Indirect Bot:** Messages that do not travel straight from the sender's server to the recipient's. 10 points for each of the following:

   - **Relaying:** your server passed at least 10 messages on towards the correct recipient, where your group is neither sender nor receiver. Point at the `SENDMSG` lines in and out.
   - **Patience:** your server accepted **at least 2 messages** for a group that was *not reachable at the time*, held them, and delivered them once it was reachable again. For each, point at the arrival, a line showing the recipient was unreachable, and the delivery.

The two can be claimed independently: the first is about routing a message somewhere, the second about keeping it until there is somewhere to route it to.

(e) (10 points) **Edge Bot:** Your server connected to at least 3 peers, **none of them instructor servers**, on 5 occasions at least 1h apart.

(f) (20 points) **Cartographer Bot:** Map the botnet beyond your own 1-hop neighbourhood, using the `SERVERS` responses your server collects. **Map as much of it as you can reach**: a `SERVERS` response tells you about servers you are not connected to, so ask your new neighbours too — a map of your own handful of peers is not what this part is for. Provide **at least three maps**, as diagrams or adjacency lists, each labelled with the time it was taken, from **at least three different days**. Add a few observations on what changed between them: which servers were always there, which came and went, which ones most connections ran through, and whether the botnet was ever split into parts that could not reach each other.

(g) (20 points) **Courier Bot:** The League's couriers are on the botnet, and they carry orders. From time to time one of our `I_…` servers will send your group a *dispatch*: an ordinary message containing a task, and a one-time token that identifies it. Carry out the task, reply the way the dispatch tells you to, and you will be sent a fragment of something larger. **5 points per dispatch completed, up to 20 points.**

Dispatches travel like any other message, and some go out only at certain times or once your server is doing certain things. **A server that is not up and reachable will not receive them**, and they cannot be collected retroactively.

Point in your `README.md` at the log lines for each dispatch and your reply. Those same lines may **also** be claimed under **Indirect Bot** or **Edge Bot** where they qualify — a dispatch you had to get relayed is relaying, and one that reached you while you held several peers is evidence of that too.

(h) (20 points) **External Bot:** Your server runs somewhere that is *not* the TSAM server, a campus machine or the VPN — from home you will probably need port forwarding on your NAT router. You may test on the TSAM server, but large parts of your log should show your server on a non-RU address. Point at lines showing your own server name against that address (in `SERVERS` responses from your peers, for instance); we will cross-check with our own logs.

(i) (20 points) **Identify the Enemy: 5 points per correctly identified instance**,

up to 20, for spotting a server that is not a well-behaved member of the League. Instances must be at least 2 hours apart to count separately. For each, give in your `README.md`:

- the IP address and port, and the time you saw it;
- the lines you base the claim on;
- **why** you believe this was not a legitimate participant — the reasoning is what we grade, not the guess.

Note that a server behaving badly is not necessarily hostile: a group whose code is broken looks much the same from outside.

Hints

Not required, not graded.

Keep your server running after you log out

Started from an ssh session, your server dies when that session ends.

```
$ nohup ./tsamserver 4044 >/dev/null 2>&1 &
[1] 48219
$ exit
```

- `nohup` detaches it from the terminal; `&` gives you your prompt back.
- **Redirect both streams.** A background process writing to a closed terminal can be stopped by the system.
- Already running? `Ctrl-Z`, then `bg`, then `disown`.
- Your `server.log` is unaffected either way — `log_event.h` writes straight to the file.
- Find it and stop it later:

  ```
  $ pgrep -a -u $USER tsamserver
  $ kill 48219
  ```

- `tmux` is the better habit: `tmux new -s a8`, then `Ctrl-B d` to leave it, `tmux attach -t a8` to come back — days later, from another machine.
- **Clean up old processes.** The whole class shares this machine, and an abandoned server still holds its port and still appears on the botnet.

Read what our servers say about yours

```
$ tail -f /tmp/I_instr1_tsam.log
```

Each of our servers keeps a world-readable log in `/tmp/` on the TSAM server, named after its group ID. It holds what happened *because of a student server*:

- connections accepted, refused or closed;
- commands we rejected, and why;
- complaints about a malformed `HELO` or `SERVERS`, or an implausible timestamp;
- **our own failed attempts to connect to you** — which you cannot see from your side at all.

Not a grade, not complete: just the last few minutes of what our servers noticed. But it is the quickest answer to “why is nothing working”.

Find out whether anyone can reach you

A server nobody connects to looks exactly like an empty botnet. You cannot tell the difference from your own side — so ask one of ours to connect to you. Send it an ordinary message (using `SENDMSG`) whose content is:

```
TEST_ME
```

It will try, and reply telling you what happened.

- **Content of `SENDMSG`, not a command.** You don’t need to implement anything extra.
- **We dial the address the message came from, it must not be relayed**.
- One attempt per group per minute, so a refusal may just mean “not yet”.

Share files with your group

Everyone has their own user *and their own group*, so **by default the others cannot read your files** — including the log.

To give `bob` and `carol` access to `~/a8`:

```
$ chmod o+x ~
$ setfacl -R    -m u:bob:rwX -m u:carol:rwX ~/a8
$ setfacl -R -d -m u:bob:rwX -m u:carol:rwX ~/a8
```

- Line 1 lets them traverse your home directory.
- Line 2 grants access to the files that exist *now*.
- **Line 3 is the one people forget.** `-d` sets a *default* ACL, so files created later — your log above all — get the same access. Without it this works, then mysteriously stops.
- `getfacl ~/a8` shows what is set. Capital `X` means “execute only where it already applies”.

Also, use **git** for your code (and maybe the server.log).

Decide in the first few days who holds the log. It has to cover the whole fortnight, and splicing two partial logs together afterwards is worse.